

5 - Python

Codice

Variabile

Una variabile in Python rappresenta una specifica locazione di memoria riservata per conservare dati di vario tipo, come interi, decimali, stringhe o oggetti complessi.

Ad essa viene associato un identificatore, ovvero un nome che segue regole precise:

- Deve iniziare con una lettera o con un underscore,
- Può contenere caratteri alfanumerici
- Non deve coincidere con nessuna parola riservata del linguaggio.

Per convenzione, i nomi delle variabili iniziano sempre con una lettera minuscola.

Rispetto ad altri linguaggi, in Python non è necessario dover specificare il tipo.

Operatori Matematici, Logici e Tipi di Dato

Python permette di visualizzare i dati a schermo tramite la funzione integrata `print()` e di eseguire calcoli su valori interi (int) e decimali (float) utilizzando i classici operatori matematici per addizione, sottrazione, moltiplicazione, divisione, modulo e potenza, supportando anche notazioni compatte come `+=` per l'incremento.

Oltre ai numeri, assumono un'importanza cruciale le variabili booleane, che possono assumere esclusivamente i valori `True` o `False`. Su di esse è possibile applicare le regole delle tavole di verità attraverso gli operatori logici `and`, `or` e `not`. I valori booleani sono spesso il risultato diretto del confronto tra variabili, effettuabile mediante operatori relazionali come l'uguaglianza (`==`), la disuguaglianza (`!=`), il minore (`<`), il maggiore (`>`) e le loro relative varianti inclusive (`<=`, `>=`).

Strutture di Controllo e Cicli Iterativi

Per implementare la logica decisionale all'interno di un algoritmo, Python utilizza i blocchi condizionali definiti dalle parole chiave `if`, `elif` ed `else`, i quali eseguono porzioni di codice diverse in base alla veridicità di uno o più predicati.

≡ Esempio di indentazione e codice sulle strutture di controllo

```
if <cond1>:
    <istruzioni per cond1 == True>
elif <cond2>:
    <istruzioni per cond1 == False e cond2 == True>
```

```
else:
    <istruzioni per cond1 == False e cond2 == False>
```

L'intero linguaggio si basa sull'indentazione (solitamente 4 spazi) per raggruppare visivamente e strutturalmente le istruzioni: un'indentazione errata non è solo un problema stilistico, ma genera comportamenti anomali o errori di esecuzione.

Quando si presenta la necessità di ripetere un'azione fintanto che una specifica condizione iniziale risulta vera, si ricorre al ciclo iterativo `while`, che valuta la condizione ad ogni passaggio e interrompe l'esecuzione nel momento in cui essa diventa falsa.

≡ Esempio di while

```
while <condizione>:
    <istruzioni fino a quando condizione == True>
else:
    <istruzioni quando condizione == False>
```

Liste e Manipolazione dei Dati

Le liste rappresentano una sequenza ordinata di oggetti di qualsiasi tipo, racchiusa tra parentesi quadre e separata da virgole, con la peculiarità di poter contenere al suo interno anche altre liste.

≡ Esempio di lista

```
l = [4.02, "bruno", x, True]
```

L'accesso agli elementi avviene tramite un sistema di indicizzazione che parte da 0 per il conteggio da sinistra verso destra, o da -1 per esplorare la lista a ritroso partendo dall'ultimo elemento.

```
print(l[0])    # 4.02  (primo elemento)
print(l[-1])   # True  (ultimo elemento)
```

Python fornisce un ricco arsenale di metodi per manipolare le liste:

- È possibile aggiungere elementi con `append()` o `extend()`,

- È possibile inserire in posizioni specifiche elementi con `insert()`, rimuoverli con `remove()` o estrarli tramite `pop()`,
- Ordinare elementi in modo crescente o decrescente con `sort()` e `reverse()` e contarne le occorrenze con `count()`.

≡ Esempio dei metodi descritti

```
x = [5]
l = [4.02, "claudio", x, True]
l.append("giulio")    # aggiunge in fondo
l.extend(x)           # estende con gli elementi di x
l.insert(2, False)    # inserisce False all'indice 2
l.remove("claudio")   # rimuove la prima occorrenza
e = l.pop(3)           # estrae e rimuove l'elemento all'indice 3
print("lista: {}, elemento: {}".format(l, e))
# Output: lista: [4.02, False, [5], "giulio", 5], elemento: True

l = [7, 22.1, 22.1, 7, 7, 11.6]
l.sort()               # ordinamento crescente
l.reverse()            # ribalta la lista
l.sort(reverse=True)   # equivalente a sort() + reverse()
print(l.index(22.1))   # 1 - indice della prima occorrenza
print(l.count(7))      # 3 - numero di occorrenze
```

Essendo oggetti mutabili, la modifica di una lista altera il contenuto del contenitore originale in memoria; per questo motivo, se si desidera creare una copia indipendente, è necessario utilizzare il metodo `.copy()` oppure ricorrere alla notazione `[start:end:step]`, che consente di estrarre porzioni specifiche di una lista definendo l'indice di partenza incluso, quello di fine escluso e il passo di avanzamento, offrendo persino la possibilità di invertire l'intera lista semplicemente impostando un passo negativo.

≡ Esempio di notazione start:end:step

```
l = [2, 3, 10, 50]
c = l[1:3]           # [3, 10]
print(l[:2])         # [2, 3]
print(l[1:])         # [3, 10, 50]
```

```
print(l[:])          # [2, 3, 10, 50]
print(l[::-1])       # [50, 10, 3, 2]  <- lista invertita!
```

Tuple, Sequenze e Range

Le liste appartengono alla più ampia famiglia delle sequenze, un raggruppamento concettuale che include anche le stringhe di testo e le tuple.

≡ Esempio di stringa come sequenza

```
s = "Hello"
print(s[2])    # 'l'
print(s[3:])   # 'lo'
```

A differenza delle liste, le tuple si definiscono tramite parentesi tonde e sono rigorosamente immutabili, prestandosi particolarmente alla conservazione di dati disomogenei che non devono subire alterazioni nel tempo.

≡ Esempio di una tupla

```
t = ("a", 2, "c")
t[1] = "b" # TypeError! Le tuple sono immutabili
```

Sarebbe raccomandabile mettere in una tupla i dati disomogenei e in una lista quelli omogenei

Tutte le sequenze condividono funzionalità comuni:

- Gli operatori `in` e `not in` permettono di verificare l'esistenza di un elemento,
- L'operatore di addizione concatena più sequenze, la moltiplicazione per un numero intero le replica.
- Funzioni integrate come `len`, `min`, `max` e `sum` ne analizzano il contenuto.

≡ Esempio di funzioni di sequenze

```
l = [None] * 100 # lista di 100 elementi None

len(l) # lunghezza
```

```
min(l)    # valore minimo
max(l)    # valore massimo
sum(l)    # somma degli elementi
```

Una caratteristica potente delle sequenze è il "sequence unpacking", che permette di assegnare i singoli valori interni a variabili distinte in una sola riga di codice.

≡ Esempio di sequence unpacking

```
l = (4, 6, 8, 10)
a, b, _, c = l
# a=4, b=6, _=8 (ignorato), c=10
```

Per generare rapidamente sequenze numeriche entra in gioco il tipo di dato `range`, utilissimo per creare contatori specificando un punto di inizio, un termine escluso e un passo, dove il parametro di partenza è considerato zero se omesso.

≡ Esempio di range

```
range(start, end, step)

# Esempio: numeri da 1 a 100 con passo 3
l = list(range(1, 100, 3))
```

Set e Dizionari

Python gestisce i dati non ordinati attraverso i set e i dizionari. Il set, delimitato da parentesi graffe, è una collezione non ordinata e non indicizzabile che garantisce l'assenza di duplicati, rivelandosi lo strumento ideale per gestire enormi moli di dati su cui effettuare rapide ricerche di appartenenza.

≡ Esempio di set

```
s = {4, 2, 4, 1}
print(s)    # {1, 2, 4} <- il duplicato è rimosso
# s[1]      # TypeError! I set non sono indicizzabili
```

Il dizionario, anch'esso definito da parentesi graffe, struttura invece le informazioni come collezioni mutabili di coppie chiave-valore.

≡ Esempio di dizionario

```
d = {'Uniba': 'Università nel Sud Italia', 'Gatto': 'Mammifero di tipo felino'}
# Oppure, se le chiavi sono identificatori validi:
d = dict(Uniba='...', Gatto='...')
```

Per accedere al valore di un dizionario è sufficiente richiamare la sua chiave, a patto che questa sia un identificatore valido, utilizzando le parentesi quadre oppure il metodo `.get()`.

≡ Esempio di accesso ai valori del dizionario

```
d = {'Uniba': 'Università nel Sud Italia', 7: 'Numero intero'}
print(d['Uniba'])          # accesso con parentesi quadre
print(d.get(7))           # accesso con .get()
d['Bari'] = 'Bellissima città' # aggiunta di una nuova coppia
```

Esattamente come accade per le liste, anche i dizionari sono oggetti mutabili, il che significa che una semplice assegnazione ne copia solo il riferimento in memoria, rendendo necessario l'uso del metodo `.copy()` per duplicare l'oggetto in modo sicuro.

≡ Esempio di mutabilità e copia dei dizionari

```
d = {'Uniba': 'Università nel Sud Italia', 7: 'Numero intero'}
c = d
c['Uniba'] = 5
print(d)    # anche d è cambiato!
```

```
# Per una vera copia:
c = d.copy()
```

Iterazioni Avanzate: Il Ciclo For

Per scorrere gli elementi di qualsiasi oggetto in grado di restituire un dato alla volta (chiamato iterabile), Python introduce il ciclo `for`. Questo costrutto può percorrere liste, tuple, stringhe, set e dizionari.

≡ Esempio di ciclo for su una lista e con range

```
seq = [1, 2, 3, 4, 5]
for item in seq:
    print(item)

# Con range come contatore:
for i in range(0, 100):
    print(i)
```

In combinazione con il ciclo `for`, funzioni come `zip()` consentono di iterare simultaneamente sugli elementi corrispondenti di iterabili diversi, mentre `enumerate()` restituisce in contemporanea sia l'elemento corrente che il suo indice posizionale.

≡ Esempio di zip() ed enumerate()

```
numbers = [1, 2, 3, 4]
letters = ['a', 'b', 'c', 'd']
for n, l in zip(numbers, letters):
    print(f'{n}: {l}')

seq = ['a', 'b', 'c']
for index, item in enumerate(seq):
    print(index)
    print(item)
```

L'iterazione sui dizionari può avvenire estraendo le sole chiavi con `.keys()` o, in maniera più efficiente, sfruttando il metodo `.items()` abbinato al sequence unpacking per ottenere

ed elaborare direttamente sia la chiave che il valore associato all'interno del blocco di iterazione.

≡ Esempio di iterazione su dizionari

```
d = {1: "Bruno", 2: "Giulio", 3: "Luigi", 4: "Fabio"}

# Iterazione sulle chiavi:
for k in d.keys():
    print("chiave {} valore {}".format(k, d[k]))

# Iterazione su coppie chiave-valore (modo più efficiente):
for key, val in d.items():
    print("chiave {} valore {}".format(key, val))
```

List Comprehension e generatori

Una delle caratteristiche più eleganti del linguaggio è la "list comprehension", una tecnica compatta per creare, filtrare e manipolare liste in una sola riga di codice sostituendo ingombranti costrutti iterativi.

≡ Esempio di list comprehension

```
x = range(1, 5)
h = [b ** 2 for b in x]                # [1, 4, 9, 16]
f = [b ** 2 for b in x if b % 2 == 0]  # [4, 16]
g = [b ** 2 if b % 2 == 0 else 0 for b in x] # [0, 4, 0, 16]
```

Questa sintassi può essere nidificata con loop interni ed esterni per elaborare strutture dati complesse, come ad esempio appiattire matrici o trasporle.

≡ Esempio di list comprehension annidata

```
# Appiattire una lista di liste con sostituzione condizionale:
planets = [['Mercurio', 'Venere', 'Terra'],
            ['Marte', 'Giove', 'Saturno'],
            ['Urano', 'Nettuno', 'Plutone']]
flat = [p if len(p) <= 6 else 'NA' for row in planets for p in row]
```



```
# Trasposto di una matrice:
matrix = [[2, 6, 5], [4, 3, 5], [6, 6, 1], [1, 2, 3]]
transposed = [[row[i] for row in matrix] for i in
range(len(matrix[0]))]
```

Quando non si desidera caricare un'intera lista in memoria ma elaborare i dati a run-time, si ricorre alle espressioni generatore (generator), che mantengono una sintassi simile alla list comprehension ma utilizzano parentesi tonde, rivelandosi essenziali per inserire flussi di dati all'interno di funzioni che convertono o sommano i tipi di dato senza spreco di risorse.

≡ Esempio di espressioni generator

```
a = sum(expression for name in seq)
b = set(expression for name in seq)
```

Funzioni e Programmazione Funzionale

Per evitare ripetizioni di codice e rendere i programmi modulari, Python permette di incapsulare blocchi logici all'interno di funzioni personalizzate tramite la parola chiave `def`, potendo specificare parametri (anche con valori di default opzionali) e restituire uno o più risultati attraverso l'istruzione facoltativa `return`.

≡ Definizione e chiamata di una funzione

```
def power(n, exp=2):
    s = n ** exp
    return s

for i in range(1, 5):
    res = power(i)
    print(res)
```

Per operazioni estremamente brevi o matematiche, è possibile dichiarare funzioni anonime note come funzioni `lambda`, che condensano l'intera logica in un'unica espressione.

≡ Funzione lambda

```
power = lambda n, exp: n ** exp
```

Queste ultime trovano la loro massima utilità nei paradigmi di programmazione funzionale, venendo spesso passate come argomenti a costrutti come `filter()`, che estrae elementi basandosi su un predicato booleano, e `map()`, che applica una trasformazione sistematica agli elementi di uno o più iterabili.

≡ `filter()` e `map()` con `lambda`

```
a = [1, 2, 3, 4, 5]
b = [9, 8, 7, 6, 5]

# filter: restituisce gli elementi di a per cui la funzione è True
filter(lambda x: x < 3, a)

# map: applica la funzione a ciascuna coppia di elementi
map(lambda x, y: x + y, a, b)
```

Moduli e Programmazione Orientata agli Oggetti (OOP)

All'aumentare della complessità del progetto, il codice viene suddiviso in file separati che possono essere integrati mediante i comandi `import`, l'assegnazione di alias o l'importazione mirata di singole funzionalità con `from ... import`.

≡ Import di moduli

```
import utilities
utilities.square(8)

import utilities as alias
alias.square(8)

from utilities import square
square(8)
```

L'apice dell'organizzazione strutturale si raggiunge con la Programmazione Orientata agli Oggetti, che modella concetti tramite classi e ne istanzia specifici individui. All'interno di una classe, il costruttore `__init__` inizializza le variabili (attributi), mentre la parola chiave `self` garantisce l'accesso alle caratteristiche e ai metodi dello specifico individuo.

≡ Definizione di una classe

```
class Studente:
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome

    def get_anagrafica(self):
        return f"Anagrafica studente\nNome: {self.nome}\nCognome: {self.cognome}"

s1 = Studente("Mario", "Rossi")
print(s1.get_anagrafica())
```

Python supporta nativamente l'ereditarietà: una classe derivata può recepire attributi e metodi da una classe genitore e accedere a quest'ultima tramite la funzione `super()`, con la libertà di sovrascrivere (overriding) le funzionalità ereditate per adattarle alle nuove esigenze.

≡ Ereditarietà e overriding

```
class Persona:
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome

    def get_anagrafica(self):
        return f"Anagrafica\nNome: {self.nome}\nCognome: {self.cognome}"

class Studente(Persona):
    def __init__(self, nome, cognome, corso_di_studio):
        super().__init__(nome, cognome)
        self.corso_di_studio = corso_di_studio
```

```
def get_anagrafica(self):
    return super().get_anagrafica() + f"\nCorso di studio:
{self.corso_di_studio}"
```

A questo si aggiunge il concetto di polimorfismo gestito dai metodi dunder (double underscore), come `__str__` o `__add__`, che permettono di ridefinire comportamenti standard del linguaggio per gli oggetti personalizzati.

≡ Metodi dunder e polimorfismo

```
class Studente:
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome

    def __str__(self):
        return f"Oggetto contenente informazioni sullo studente
{self.nome} {self.cognome}"

x = Studente("Mario", "Rossi")
print(x) # chiama automaticamente __str__
```

Infine, l'incapsulamento e la sicurezza dei dati si ottengono per convenzione tramite l'uso di un singolo underscore per marcare attributi protetti (ereditabili ma logicamente interni), o due underscore consecutivi per definire attributi rigorosamente privati, nascosti all'esterno e alle classi figlie.

≡ Attributi pubblici, protetti e privati

```
class Test:
    def __init__(self):
        self.a = 11 # pubblico
        self._b = 23 # protetto (convenzione)
        self.__c = 23 # privato (name mangling)
```